

LAB 2

Styling: Tailwind CSS and shadcn/ui

Course “Advanced Web Technologies” · Week 3 · 1 contact hour + independent completion before the next class

Goal

Style the course catalog you've built in the previous labs using Tailwind CSS and shadcn/ui — and make the interface responsive: clean on phone, tablet, and desktop alike. No new logic today, only the look of an already-working project.

Before you start

- a working course-catalog project from Lab 1 — it opens and starts with `npm run dev`;
- Lecture 3's handout and slides open nearby — today you'll follow section 7 almost word for word;
- no design perfectionism required — looks aren't graded, working correctly and responsively is.

What you'll build

The same course catalog as before, but now with shadcn/ui cards instead of bare divs, a responsive grid for the course list, and dark mode support if you choose the bonus track:

```
before / after
before: <div className="">{title}</div>
after:  <Card className="hover:shadow-md transition">
        <CardHeader><CardTitle>{title}</CardTitle></CardHeader>
        ...
      </Card>
```

1. Installing shadcn/ui

In your course-catalog project folder:

```
terminal
npx shadcn@latest init
```

Answer the wizard's questions with the default values — they're already consistent with what create-next-app set up in Lab 1. Next, add the components you'll need today:

```
terminal
npx shadcn@latest add button card
```

If something goes wrong: make sure you're inside the course-catalog folder (not one level up), and that it's best to stop `npm run dev` for a moment — some versions of the wizard conflict with a running dev server.

After this command, `components/ui/button.tsx` and `components/ui/card.tsx` will appear in your project — open them and skim through, you'll need them in section 2.

Check yourself: a `components.json` file should now exist at the project root, and `components/ui/` should contain `button.tsx` and `card.tsx`. If either is missing, the add command didn't succeed — run it again and watch the terminal output.

2. Rewriting CourseCard

The task is to replace the contents of `components/CourseCard.tsx` so it uses `Card` and `Button` from `shadcn/ui` instead of plain HTML tags. The props contract doesn't change — it's the same `CourseCardProps` from Lab 1.

```
components/CourseCard.tsx – imports you'll need
import Link from "next/link";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Button } from "@components/ui/button";
```

- the whole card is still wrapped in `<Link href={` /courses/${id} `}>`, but now it contains a `<Card>`, not a `<div>`;
- the course title goes inside `<CardHeader><CardTitle>`;
- the description and the bottom row with credits and likes go inside `<CardContent>`;
- the like button is a `<Button variant="ghost" size="sm">` component, not a plain `<button>` or bare text;
- add `hover:shadow-md hover:border-blue-300 transition` to the Card's own `className` — this gives a subtle hover effect.

If you're not sure how to start the function, here's a skeleton with every component already in place — you just need to fill in `title/description/credits/likes`:

```
components/CourseCard.tsx – skeleton
function CourseCard({ id, title, description, credits, likes }: CourseCardProps) {
  return (
    <Link href={` /courses/${id} `}>
      <Card className="hover:shadow-md hover:border-blue-300 transition">
        <CardHeader>
          <CardTitle className="text-lg">{/* TODO: title */}</CardTitle>
        </CardHeader>
        <CardContent className="flex flex-col gap-3">
          {/* TODO: description */}
          <div className="flex items-center justify-between">
            {/* TODO: credits */}
            <Button variant="ghost" size="sm">{/* TODO: ❤️ + likes */}</Button>
          </div>
        </CardContent>
      </Card>
    </Link>
  );
}
```

Why structure it this way: `CardHeader` and `CardContent` aren't just wrappers for looks — they already carry Tailwind spacing that's consistent with each other. If you put everything into a single `CardContent` with no `CardHeader`, the layout still works, but loses some of the vertical rhythm `shadcn/ui` builds in by default.

Hint: if the card won't type-check or complains about its props — check it against the `CourseCard` example in section 7 of the Lecture 3 handout and the final-card slide in the deck; it walks through exactly this pattern.

The component stays a Server Component — don't add 'use client' to CourseCard, and Card/Button don't need it either.

3. A Responsive Grid for the Course List

Open app/courses/page.tsx. Right now the course list most likely renders as a single column. Wrap it in a container with responsive Tailwind classes:

```
app/courses/page.tsx - fragment
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
  {courses.map((course) => (
    <CourseCard key={course.id} {...course} />
  ))}
</div>
```

Why these three steps specifically? grid-cols-1 with no prefix is what a phone sees by default (section 3 of the lecture: mobile-first). sm:grid-cols-2 kicks in at 640px — a typical tablet width in portrait orientation. lg:grid-cols-3 kicks in at 1024px, already a desktop. None of this needs memorizing — it's the same logic as the responsive-diagram slide in the deck.

Want to check that you actually understood the principle, not just copied the line? Try it yourself — without peeking — add a fourth step: xl:grid-cols-4 for very wide monitors (1280px and up). The result should come out as one short edit to the same line.

4. Styling the Navigation

In app/layout.tsx you should already have simple navigation from Lab 1 (Home / Courses / About) — most likely three bare Link components in a row with no classes at all. Let's tidy it up now.

```
app/layout.tsx - navigation skeleton
<nav className="flex gap-4 px-6 py-4 border-b border-slate-200">
  <Link href="/" className="px-3 py-2 rounded-md hover:bg-slate-100">
    Home
  </Link>
  <Link href="/courses" className="px-3 py-2 rounded-md hover:bg-slate-100">
    Courses
  </Link>
  <Link href="/about" className="px-3 py-2 rounded-md hover:bg-slate-100">
    About
  </Link>
</nav>
```

Breaking down the classes: flex lines the three links up in a row (without it they'd stack vertically, like normal block elements); gap-4 is the space between links; px-3 py-2 is the padding inside each link, so the clickable area is bigger than the text itself; border-b is a thin line underneath, separating the nav from the page content.

The values px-3, py-2, gap-4 aren't the only correct ones — they're a starting point. These must be present:

- spacing between links and inside each link — so the clickable area is bigger than the text alone;

- a visible hover state (hover:bg-slate-100, hover:text-blue-600 — your choice);
- visual separation between the nav and the rest of the page content (border-b, a shadow, or a different background).

Optional, but nice: if you'd like to highlight the current page — compare the current path via the `usePathname()` hook from `next/navigation` against each link's `href`, and add a separate class for a match. This already requires 'use client' in the nav component — move it into its own `components/NavBar.tsx` file so you don't have to make all of `layout.tsx` a client component.

5. Actually Verifying Responsiveness

Your instructor's wide monitor will show three columns even if there's a typo in your code — screen size alone will hide the bug. Before submitting, check all three widths explicitly, using your browser's developer tools:

1. open the `/courses` page in your browser and press F12 (or right-click → “Inspect”);
2. turn on responsive/device preview mode — the phone-and-tablet icon in the top-left of the dev tools panel (Chrome and Edge) or `Ctrl+Shift+M`;
3. pick a phone (e.g. iPhone SE), a tablet (iPad), and “Responsive” at around 1280px width, one at a time, from the device list;
4. at each width, confirm the number of columns matches what's intended, and that cards don't overlap or get cut off.

Acceptance Checklist

Before you submit, run through this list:

- ☐ `npm run dev` starts up with zero TypeScript errors in the terminal;
- ☐ `npm run build` completes successfully;
- ☐ `CourseCard` uses `Card`, `CardHeader`, `CardTitle`, `CardContent`, and `Button` from `shadcn/ui` — not a single bare `div/button` remains;
- ☐ the course list at `/courses` shows 1 column on narrow screens, 2 on medium, 3 on wide — verified through DevTools, not just eyeballed;
- ☐ cards have a visible hover effect;
- ☐ the nav in `layout.tsx` is styled, laid out in a row, and has a hover state;
- ☐ `components/CourseCard.tsx` is still without 'use client' — the component stays a server component;
- ☐ changes are committed to the same GitHub repository as Lab 1.

Grading Rubric

Criterion	Points
shadcn/ui installed, Button and Card added	1

Criterion	Points
CourseCard rewritten with Card/CardHeader/CardTitle/CardContent/Button	3
Course list is responsive: 1 / 2 / 3 columns by screen width	2
Cards have a hover effect	1
Navigation is styled and has a hover state	1
CourseCard stays a Server Component (no unnecessary 'use client')	1
Changes committed to the repository with meaningful messages	1

Total: 10 points. Adjust the scale to your own grading system as needed.

Common Mistakes

'use client' on the card: Card and Button from shadcn/ui don't require client rendering on their own — if TypeScript or React complains, look for the cause somewhere else rather than adding the directive “just in case.”

Grid inside CourseCard instead of outside: grid-cols-* belongs on the list container in app/courses/page.tsx, not inside the CourseCard component itself — otherwise the responsiveness won't work.

A missing npx shadcn add: if the import from "@components/ui/card" can't be found — the component probably wasn't added. Check that components/ui/card.tsx actually exists in the project.

Checking responsiveness only on your own monitor: a wide monitor will almost always show 3 columns regardless of mistakes in your classNames — make sure to go through section 5 with DevTools before submitting.

Bonus Tasks (optional)

If the core part is done early — pick any, no limit on how many:

1. add your own color via the @theme block in app/globals.css and use it in at least one place in the interface (e.g. for an accent button);
2. add dark mode support for CourseCard and the navigation using the dark: prefix — no need for a click-to-toggle switch, following the system setting is enough;
3. open components/ui/button.tsx, study the existing variants, and add your own custom style variant;
4. add a shadcn/ui component that wasn't covered in the lecture (e.g. badge or skeleton for a loading state), and use it somewhere it fits in the project.

How to Submit

Push your changes to the same course-catalog repository as your previous labs.

In README.md, add a short line about what was styled in this lab.

Send the repository link (or just a message saying “done, commits pushed”) however your instructor specifies.

Deadline: before Lecture 4 begins, unless your instructor states otherwise.